

Reflection: Date Display GUI Application with MVC Pattern

Design Decisions and the Rationale For the Date Display application, the primary design decision was implementing the Model-View-Controller (MVC) architectural pattern. The rationale behind this approach is to ensure a strict separation of concerns, which is essential for building scalable and maintainable software. The DateModel was designed to exclusively encapsulate the application state and business logic regarding date manipulations. The DateView is strictly responsible for rendering the graphical user interface using Java Swing components and capturing user inputs without processing them. Finally, the DateController serves as the crucial bridge, listening to user actions from the view, updating the model based on those inputs, and subsequently triggering the view to refresh its display.

Main Functionalities The application automatically retrieves and displays the current date in a Vietnamese-style format (dd/MM/yyyy) upon startup. Its core functionalities allow users to input a specific number and navigate the displayed date forward or backward by either days or months. The application safely handles chronological edge cases such as leap years and months with varying numbers of days. Additionally, it features robust error handling functionalities; if a user enters a negative number, non-numeric characters, or leaves the input field empty, the system successfully intercepts the invalid input and displays a clear error dialog to prevent application crashes.

Brief Explanation of the Implementation The implementation utilizes Java's built-in `java.time.LocalDate` class to manage complex date arithmetic safely and accurately. A `DateTimeFormatter` is used within the view to ensure consistent formatting. Following the MVC workflow, the DateController attaches an `ActionListener` to the interactive buttons provided by the DateView. Upon a button click event, the controller retrieves the text input, parses it within a try-catch block to handle any `NumberFormatException` or custom negative number validation. Upon valid input, the controller invokes the respective arithmetic methods (`addDays`, `minusMonths`, etc.) in the DateModel, and immediately calls the `update` method on the view to reflect the newly calculated date on the screen.